EDGVoIP API v2 - Complete Integration Documentation

Version: 2.0
Base URL: `https://edgvoip.it/api/v2/voip`
Last Updated: 2025-11-30

Table of Contents

1. [Authentication & API Keys](#authentication--api-keys)
2. [Identifiers & Relationships](#identifiers--relationships)
3. [Stores API](#stores-api)
4. [SIP Trunks API](#sip-trunks-api)
5. [Extensions API](#extensions-api)
6. [Status Monitoring API](#status-monitoring-api)
7. [API Logs API](#api-logs-api)
8. [Webhooks API](#webhooks-api)
9. [Health Check API](#health-check-api)
10. [CDR API](#cdr-api)
11. [Account Info API](#account-info-api)
12. [DID API](#did-api)
13. [Error Handling](#error-handling)
14. [Rate Limiting](#rate-limiting)
15. [Database Schema for W3 Suite](#database-schema-for-w3-suite)

Authentication & API Keys

Overview

EDGVoIP API v2 uses **API Key authentication for all requests. Each tenant has its own API keys that are **centralized and managed per tenant**. API keys are scoped with granular permissions.**

Creating API Keys

API keys are created through the **Developer Settings** dashboard in EDGVoIP:

1. Navigate to: `https://edgvoip.it/{tenantSlug}/developer`
2. Go to **API Keys** tab
3. Click **Generate New Key**
4. Enter a name (e.g., "W3 Suite Integration")
5. Select required scopes (permissions)
6. **Copy the key immediately** - it's shown only once!

API Key Format

sk_live_{48_random_hex_characters}

Example:
`sk_live_a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6q7r8s9t0u1v2w3x4y5z6`

Using API Keys

Include the API key in the `X-API-Key` header for **all API v2 requests**:

```
curl -X GET https://edgvoip.it/api/v2/voip/trunks   -H "X-A
```

Available Scopes

Scope	Description	Endpoints
`voip:read`	Read VoIP data (trunks, extensions, status)	GET /trunks, GET /extensions, GET /status/*
`voip:write`	Create/Update/Delete trunks	POST /trunks, PUT /trunks/:id, DELETE /trunks/:id
`extensions:write`	Create/Update/Delete extensions	POST /extensions, PUT /extensions/:id, DELETE /extensions/:id

`extensions:sync`	Sync extensions bidirectionally	POST /extensions/sync
`trunks:sync`	Sync trunks bidirectionally	POST /trunks/sync
`stores:read`	Read stores (READ ONLY)	GET /stores
`stores:sync`	Sync stores from external system	POST /stores/sync
`ai:config`	Configure AI agents on trunks	(Internal use)

API Key Management Endpoints

Note: These endpoints require JWT authentication (not API Key), accessed through Developer Settings UI.

- `GET /api/api-keys` - List all API keys for tenant
- `POST /api/api-keys` - Create new API key
- `DELETE /api/api-keys/:id` - Revoke API key

Identifiers & Relationships

Tenant External ID

Field: `tenant_external_id`
Type: VARCHAR(255)
Format: Alphanumeric (a-z, A-Z, 0-9), dots (.), hyphens (-), underscores (_) only
Required: Yes (for API requests)
Configurable: Yes, in Developer Settings > Integration tab

This is your **custom identifier** for the tenant in W3 Suite. It's used in all API requests to identify the tenant.

Example: `w3suite-tenant-001`

Configuration:
1. Go to Developer Settings > Integration tab
2. Enter your custom `tenant_external_id`
3. Save

Store ID

Field: `store_id`

Type: VARCHAR(255)
Format: Custom (any string)
Required: Yes (for trunks, optional for extensions)

This is your **custom identifier** for stores. Managed in EDGVoIP Stores page.

External ID

Field: `external_id`
Type: UUID
Format: Standard UUID (e.g., `550e8400-e29b-41d4-a716-446655440000`)
Required: Yes (auto-generated if not provided)
Usage: Used in API URLs to identify resources

Important:
- If you create a resource via API and don't provide `external_id`, EDGVoIP auto-generates one (UUID)
- If you provide `external_id`, EDGVoIP uses it (must be valid UUID)
- `external_id` is used in all API URLs:
`/api/v2/voip/trunks/:external_id`

Relationship Diagram

```
Tenant (tenant_external_id: "w3suite-001")
├── Store (store_id: "store-001")
│   ├── SIP Trunk (external_id: UUID)
│   └── Extension (external_id: UUID)
└── Extension (external_id: UUID, store_id: null)
```

Stores API

Base Endpoint
`/api/v2/voip/stores`

GET /api/v2/voip/stores

Description: List all stores for the tenant (READ ONLY)

Authentication: `stores:read` scope required

Request:

```
curl -X GET https://edgvoip.it/api/v2/voip/stores   -H "X-A
```

Response:
json
{
"success": true,
"data": [
{
"id": "550e8400-e29b-41d4-a716-446655440000",
"store_id": "store-001",
"tenant_external_id": "w3suite-001",
"name": "Main Store",
"status": "active",
"settings": {},
"created_at": "2025-11-30T10:00:00Z",
"updated_at": "2025-11-30T10:00:00Z"
}
],
"meta": {
"total": 1,
"timestamp": "2025-11-30T12:00:00Z"
}
}

Response Fields:
- `id` (UUID): Internal EDGVoIP ID
- `store_id` (string): Custom store identifier (your ID)
- `tenant_external_id` (string): Tenant custom identifier
- `name` (string): Store name
- `status` (string): `active` | `inactive`

- `settings` (object): Store settings
- `created_at` (ISO 8601): Creation timestamp
- `updated_at` (ISO 8601): Last update timestamp

GET /api/v2/voip/stores/:store_id

Description: Get store by custom `store_id`

Authentication: `stores:read` scope required

Request:

```
curl -X GET https://edgvoip.it/api/v2/voip/stores/store-001
```

Response:
json
{
"success": true,
"data": {
"id": "550e8400-e29b-41d4-a716-446655440000",
"store_id": "store-001",
"tenant_external_id": "w3suite-001",
"name": "Main Store",
"status": "active",
"settings": {},
"created_at": "2025-11-30T10:00:00Z",
"updated_at": "2025-11-30T10:00:00Z"
}
}

Error Response (404):
json
{
"success": false,
"error": "Store not found",
"error_code": "STORE_NOT_FOUND"

}

POST /api/v2/voip/stores/sync

Description: Sync stores from W3 Suite to EDGVoIP (EDGVoIP is child, only reads)

Authentication: `stores:sync` scope required

Request:

```
curl -X POST https://edgvoip.it/api/v2/voip/stores/sync   -
    "force": false,
    "store_ids": []
  }'
```

Request Body:
json
{
"force": false,
"store_ids": []
}

Fields:
- `force` (boolean, optional): Force full sync
- `store_ids` (array, optional): Specific store IDs to sync (empty = all)

Response:
json
{
"success": true,
"synced": 5,
"created": 0,
"updated": 0,
"errors": [],
"stores": [

{
"id": "550e8400-e29b-41d4-a716-446655440000",
"store_id": "store-001",
"tenant_external_id": "w3suite-001",
"name": "Main Store",
"status": "active"
}
]
}

Note: EDGVoIP never creates stores via API. Stores must be created in EDGVoIP UI first.

SIP Trunks API

Base Endpoint
`/api/v2/voip/trunks`

GET /api/v2/voip/trunks

Description: List all SIP trunks for the tenant

Authentication: `voip:read` scope required

Query Parameters:
- `store_id` (optional): Filter by store_id

Request:

```
curl -X GET "https://edgvoip.it/api/v2/voip/trunks?store_id
```

Response:
json
{
"success": true,

"data": [
{
"id": "550e8400-e29b-41d4-a716-446655440000",
"external_id": "550e8400-e29b-41d4-a716-446655440000",
"tenant_external_id": "w3suite-001",
"store_id": "store-001",
"name": "Warian Trunk",
"provider": "warian",
"status": "active",
"sip_config": {
"host": "sip.warian.it",
"port": 5060,
"transport": "udp",
"username": "user123",
"password": "***REDACTED***",
"realm": "sip.warian.it",
"from_user": "user123",
"from_domain": "sip.warian.it",
"register": true,
"register_proxy": null,
"register_transport": "udp",
"retry_seconds": 60,
"caller_id_in_from": true,
"contact_params": null,
"ping": true,
"ping_time": 30
},
"did_config": {
"number": "0687165699",
"country_code": "39",
"area_code": "06",
"local_number": "87165699",
"provider_did": null,
"inbound_route": null
},
"security": {
"encryption": "none",
"authentication": "digest",
"acl": [],
"rate_limit": {
"enabled": false,

"calls_per_minute": 0,
"calls_per_hour": 0
}
},
"gdpr": {
"data_retention_days": 90,
"recording_consent_required": false,
"data_processing_purpose": "Business communications",
"lawful_basis": "contract",
"data_controller": "Company Name",
"dpo_contact": null
},
"codec_preferences": ["PCMU", "PCMA", "G729"],
"created_at": "2025-11-30T10:00:00Z",
"updated_at": "2025-11-30T10:00:00Z"
}
],
"meta": {
"total": 1,
"timestamp": "2025-11-30T12:00:00Z"
}
}

GET /api/v2/voip/trunks/:external_id

Description: Get trunk by `external_id`

Authentication: `voip:read` scope required

Request:

```
curl -X GET https://edgvoip.it/api/v2/voip/trunks/550e8400-
```

Response: Same structure as list, but single object in `data` field.

POST /api/v2/voip/trunks

Description: Create a new SIP trunk

Authentication: `voip:write` scope required

Request:

```
curl -X POST https://edgvoip.it/api/v2/voip/trunks   -H "X-
    "tenant_external_id": "w3suite-001",
    "external_id": "550e8400-e29b-41d4-a716-446655440000",
    "store_id": "store-001",
    "name": "New Trunk",
    "provider": "custom",
    "sip_config": {
      "host": "sip.provider.com",
      "port": 5060,
      "transport": "udp",
      "username": "user123",
      "password": "secret123",
      "realm": "sip.provider.com",
      "from_user": "user123",
      "from_domain": "sip.provider.com",
      "register": true,
      "register_proxy": null,
      "register_transport": "udp",
      "retry_seconds": 60,
      "caller_id_in_from": true,
      "contact_params": null,
      "ping": true,
      "ping_time": 30
    },
    "did_config": {
      "number": "1234567890",
      "country_code": "39",
      "area_code": "06",
      "local_number": "1234567890",
      "provider_did": null,
      "inbound_route": null
    },
    "security": {
      "encryption": "none",
      "authentication": "digest",
      "acl": [],
      "rate_limit": {
        "enabled": false,
        "calls_per_minute": 0,
```

```
          "calls_per_hour": 0
        }
      },
      "gdpr": {
        "data_retention_days": 90,
        "recording_consent_required": false,
        "data_processing_purpose": "Business communications",
        "lawful_basis": "contract",
        "data_controller": "Company Name"
      },
      "codec_preferences": ["PCMU", "PCMA", "G729"]
    }'
```

Request Body Structure:
json
{
"tenant_external_id": "string (REQUIRED)",
"external_id": "UUID (optional, auto-generated if not provided)",
"store_id": "string (REQUIRED)",
"name": "string (REQUIRED)",
"provider": "string (optional, default: 'custom')",
"sip_config": {
"host": "string (REQUIRED)",
"port": "number (REQUIRED)",
"transport": "udp | tcp | tls (REQUIRED)",
"username": "string (REQUIRED)",
"password": "string (REQUIRED)",
"realm": "string (optional)",
"from_user": "string (optional)",
"from_domain": "string (optional)",
"register": "boolean (REQUIRED)",
"register_proxy": "string (optional)",
"register_transport": "udp | tcp | tls (optional)",
"retry_seconds": "number (REQUIRED)",
"caller_id_in_from": "boolean (REQUIRED)",
"contact_params": "string (optional)",
"ping": "boolean (REQUIRED)",
"ping_time": "number (REQUIRED)"
},
"did_config": {

```
"number": "string (REQUIRED)",
"country_code": "string (REQUIRED)",
"area_code": "string (optional)",
"local_number": "string (REQUIRED)",
"provider_did": "string (optional)",
"inbound_route": "string (optional)"
},
"security": {
"encryption": "none | tls | srtp (optional)",
"authentication": "none | digest | tls (optional)",
"acl": "array of strings (optional)",
"rate_limit": {
"enabled": "boolean (optional)",
"calls_per_minute": "number (optional)",
"calls_per_hour": "number (optional)"
}
},
"gdpr": {
"data_retention_days": "number (optional, default: 90)",
"recording_consent_required": "boolean (optional)",
"data_processing_purpose": "string (optional)",
"lawful_basis": "consent | contract | legitimate_interest (optional)",
"data_controller": "string (optional)",
"dpo_contact": "string (optional)"
},
"codec_preferences": "array of strings (optional, default: ['PCMU', 'PCMA',
'G729'])"
}
```

Response (201 Created):

```
json
{
"success": true,
"data": {
"id": "550e8400-e29b-41d4-a716-446655440000",
"external_id": "550e8400-e29b-41d4-a716-446655440000",
"tenant_external_id": "w3suite-001",
"store_id": "store-001",
"name": "New Trunk",
"provider": "custom",
```

"status": "testing",
"sip_config": { /* ... */ },
"did_config": { /* ... */ },
"security": { /* ... */ },
"gdpr": { /* ... */ },
"codec_preferences": ["PCMU", "PCMA", "G729"],
"created_at": "2025-11-30T12:00:00Z",
"updated_at": "2025-11-30T12:00:00Z"
}
}

Error Responses:

400 Bad Request (Validation Error):
json
{
"success": false,
"error": "Missing required fields: name, sip_config, did_config",
"error_code": "VALIDATION_ERROR"
}

404 Not Found (Store not found):
json
{
"success": false,
"error": "Store not found",
"error_code": "STORE_NOT_FOUND"
}

PUT /api/v2/voip/trunks/:external_id

Description: Update an existing SIP trunk

Authentication: `voip:write` scope required

Request:

```
curl -X PUT https://edgvoip.it/api/v2/voip/trunks/550e8400-
```

```
    "name": "Updated Trunk Name",
    "sip_config": {
      "host": "new.sip.provider.com",
      "port": 5060,
      "transport": "udp",
      "username": "user123",
      "password": "newpassword",
      "register": true,
      "retry_seconds": 60,
      "caller_id_in_from": true,
      "ping": true,
      "ping_time": 30
    }
  }'
```

Request Body: All fields are optional. Only include fields you want to update.

Response:
json
{
"success": true,
"data": {
"id": "550e8400-e29b-41d4-a716-446655440000",
"external_id": "550e8400-e29b-41d4-a716-446655440000",
"tenant_external_id": "w3suite-001",
"store_id": "store-001",
"name": "Updated Trunk Name",
/* ... other fields ... */
"updated_at": "2025-11-30T12:30:00Z"
}
}

DELETE /api/v2/voip/trunks/:external_id

Description: Delete a SIP trunk

Authentication: `voip:write` scope required

Request:

```
curl -X DELETE https://edgvoip.it/api/v2/voip/trunks/550e84
```

Response:
json
{
"success": true,
"message": "Trunk deleted successfully"
}

POST /api/v2/voip/trunks/sync

Description: Sync trunks bidirectionally between W3 Suite and EDGVoIP

Authentication: `trunks:sync` scope required

Sync Strategy: EDGVoIP wins (last_updated strategy)

Request:

```
curl -X POST https://edgvoip.it/api/v2/voip/trunks/sync   -
    "tenant external_id": "w3suite-001",
    "force": false,
    "trunk_ids": []
  }'
```

Request Body:
json
{
"tenant_external_id": "string (REQUIRED)",
"force": "boolean (optional, default: false)",

"trunk_ids": "array of external_ids (optional, empty = all)"
}

Response:
json
{
"success": true,
"synced": 10,
"created": 2,
"updated": 5,
"errors": [],
"trunks": [
{
"id": "550e8400-e29b-41d4-a716-446655440000",
"external_id": "550e8400-e29b-41d4-a716-446655440000",
"name": "Trunk Name",
"status": "active"
}
]
}

Sync Logic:
1. W3 Suite sends list of trunks with `external_id`
2. EDGVoIP compares with existing trunks
3. If `external_id` exists in EDGVoIP:
- Compare `updated_at` timestamps
- EDGVoIP wins (last update)
- Update EDGVoIP trunk if W3 Suite is newer
4. If `external_id` doesn't exist:
- Create new trunk in EDGVoIP with provided `external_id`
5. If EDGVoIP has trunk not in W3 Suite list:
- Optionally delete (based on `force` flag)

Extensions API

Base Endpoint
`/api/v2/voip/extensions`

GET /api/v2/voip/extensions

Description: List all extensions for the tenant

Authentication: `voip:read` scope required

Query Parameters:
- `store_id` (optional): Filter by store_id

Request:

```
curl -X GET "https://edgvoip.it/api/v2/voip/extensions?stor
```

Response:
json
{
"success": true,
"data": [
{
"id": "550e8400-e29b-41d4-a716-446655440000",
"external_id": "550e8400-e29b-41d4-a716-446655440000",
"tenant_external_id": "w3suite-001",
"store_id": "store-001",
"extension": "2000",
"display_name": "John Doe",
"status": "active",
"type": "user",
"settings": {
"voicemail_enabled": true,
"call_forwarding": {
"enabled": false,
"destination": null
},
"recording": {
"enabled": true,
"mode": "always"
}

},
"is_registered": true,
"created_at": "2025-11-30T10:00:00Z",
"updated_at": "2025-11-30T10:00:00Z"
}
],
"meta": {
"total": 1,
"timestamp": "2025-11-30T12:00:00Z"
}
}

GET /api/v2/voip/extensions/:external_id

Description: Get extension by `external_id`

Authentication: `voip:read` scope required

Request:

```
curl -X GET https://edgvoip.it/api/v2/voip/extensions/550e8
```

Response: Same structure as list, but single object.

POST /api/v2/voip/extensions

Description: Create a new extension

Authentication: `extensions:write` scope required

Request:

```
curl -X POST https://edgvoip.it/api/v2/voip/extensions   -H
    "tenant external id": "w3suite-001",
    "external id": "550e8400-e29b-41d4-a716-446655440000",
    "store_id": "store-001",
```

```
    "extension": "2001",
    "display_name": "Jane Smith",
    "password": "securepassword123",
    "type": "user",
    "settings": {
      "voicemail_enabled": true,
      "call_forwarding": {
        "enabled": false
      },
      "recording": {
        "enabled": true,
        "mode": "always"
      }
    }
  }'
```

Request Body Structure:
json
{
"tenant_external_id": "string (REQUIRED)",
"external_id": "UUID (optional, auto-generated if not provided)",
"store_id": "string (optional)",
"extension": "string (REQUIRED, e.g., '2000')",
"display_name": "string (REQUIRED)",
"password": "string (REQUIRED, SIP password)",
"type": "user | queue | conference (optional, default: 'user')",
"settings": {
"voicemail_enabled": "boolean (optional, default: true)",
"call_forwarding": {
"enabled": "boolean (optional, default: false)",
"destination": "string (optional)"
},
"recording": {
"enabled": "boolean (optional, default: true)",
"mode": "always | on_demand (optional, default: 'always')"
}
}
}

Response (201 Created):
json
```
{
"success": true,
"data": {
"id": "550e8400-e29b-41d4-a716-446655440000",
"external_id": "550e8400-e29b-41d4-a716-446655440000",
"tenant_external_id": "w3suite-001",
"store_id": "store-001",
"extension": "2001",
"display_name": "Jane Smith",
"status": "active",
"type": "user",
"settings": { /* ... */ },
"created_at": "2025-11-30T12:00:00Z",
"updated_at": "2025-11-30T12:00:00Z"
}
}
```

Error Responses:

400 Bad Request:
json
```
{
"success": false,
"error": "Missing required fields: extension, display_name, password",
"error_code": "VALIDATION_ERROR"
}
```

404 Not Found (Store):
json
```
{
"success": false,
"error": "Store not found",
"error_code": "STORE_NOT_FOUND"
}
```

PUT /api/v2/voip/extensions/:external_id

Description: Update an existing extension

Authentication: `extensions:write` scope required

Request:

```
curl -X PUT https://edgvoip.it/api/v2/voip/extensions/550e8
    "display_name": "Updated Name",
    "password": "newpassword123",
    "settings": {
      "voicemail_enabled": false
    }
  }'
```

Request Body: All fields optional. Only include fields to update.

Response: Same as GET response with updated fields.

DELETE /api/v2/voip/extensions/:external_id

Description: Delete an extension

Authentication: `extensions:write` scope required

Request:

```
curl -X DELETE https://edgvoip.it/api/v2/voip/extensions/55
```

Response:
json
{
"success": true,
"message": "Extension deleted successfully"
}

POST /api/v2/voip/extensions/sync

Description: Sync extensions bidirectionally

Authentication: `extensions:sync` scope required

Request:

```
curl -X POST https://edgvoip.it/api/v2/voip/extensions/sync
    "tenant_external_id": "w3suite-001",
    "force": false,
    "extension_ids": []
  }'
```

Request Body:
json
{
"tenant_external_id": "string (REQUIRED)",
"force": "boolean (optional, default: false)",
"extension_ids": "array of external_ids (optional, empty = all)"
}

Response:
json
{
"success": true,
"synced": 20,
"created": 3,
"updated": 8,
"errors": [],
"extensions": [
{
"id": "550e8400-e29b-41d4-a716-446655440000",
"external_id": "550e8400-e29b-41d4-a716-446655440000",
"extension": "2000",
"display_name": "John Doe",
"status": "active"

}
]
}

Status Monitoring API

Base Endpoint
`/api/v2/voip/status`

GET /api/v2/voip/status/trunks

Description: Get real-time status of all trunks

Authentication: `voip:read` scope required

Request:

```
curl -X GET https://edgvoip.it/api/v2/voip/status/trunks
```

Response:
json
{
"success": true,
"data": [
{
"id": "550e8400-e29b-41d4-a716-446655440000",
"external_id": "550e8400-e29b-41d4-a716-446655440000",
"name": "Warian Trunk",
"status": "registered",
"registration_status": "REGED",
"health_status": "active",
"current_calls": 2,
"max_concurrent_calls": 10,
"last_check": "2025-11-30T12:00:00Z",
"last_successful_registration": "2025-11-30T11:55:00Z"

}
],
"meta": {
"total": 1,
"timestamp": "2025-11-30T12:00:00Z"
}
}

Status Values:
- `registered` : Trunk is registered and active
- `unregistered` : Trunk is not registered
- `failed` : Registration failed
- `unknown` : Status unknown

GET /api/v2/voip/status/trunks/:external_id

Description: Get status of single trunk

Request:

```
curl -X GET https://edgvoip.it/api/v2/voip/status/trunks/55
```

Response: Single trunk status object.

GET /api/v2/voip/status/extensions

Description: Get real-time status of all extensions

Authentication: `voip:read` scope required

Request:

```
curl -X GET https://edgvoip.it/api/v2/voip/status/extension
```

Response:
json
{
"success": true,
"data": [
{
"id": "550e8400-e29b-41d4-a716-446655440000",
"external_id": "550e8400-e29b-41d4-a716-446655440000",
"extension": "2000",
"display_name": "John Doe",
"status": "registered",
"connection_type": "sip",
"last_seen": "2025-11-30T12:00:00Z",
"registration": {
"ip": "192.168.1.100",
"port": 5060,
"type": "sip"
}
}
],
"meta": {
"total": 1,
"timestamp": "2025-11-30T12:00:00Z"
}
}

Status Values:
- `registered` : Extension is registered
- `unregistered` : Extension is not registered

Connection Types:
- `sip` : Standard SIP (UDP/TCP)
- `wss` : WebSocket Secure
- `webrtc` : WebRTC

GET /api/v2/voip/status/extensions/:external_id

Description: Get status of single extension

Request:

```
curl -X GET https://edgvoip.it/api/v2/voip/status/extension
```

Response: Single extension status object.

API Logs API

Base Endpoint
`/api/v2/voip/logs`

GET /api/v2/voip/logs

Description: Get API request logs with filters

Authentication: `voip:read` scope required

Query Parameters:
- `status` (optional): `success` | `error` | `warning`
- `start_date` (optional): ISO 8601 date
- `end_date` (optional): ISO 8601 date
- `endpoint` (optional): Filter by endpoint path
- `limit` (optional, default: 50): Number of results
- `offset` (optional, default: 0): Pagination offset

Request:

```
curl -X GET "https://edgvoip.it/api/v2/voip/logs?status=err
```

Response:
json

{
"success": true,
"data": [
{
"id": "550e8400-e29b-41d4-a716-446655440000",
"tenant_id": "550e8400-e29b-41d4-a716-446655440001",
"api_key_id": "550e8400-e29b-41d4-a716-446655440002",
"endpoint": "/api/v2/voip/trunks",
"method": "GET",
"request_id": "req-123456",
"status_code": 200,
"status": "success",
"request_body": { /* sanitized, passwords removed */ },
"response_body": { /* sanitized */ },
"error_message": null,
"error_code": null,
"execution_time_ms": 45,
"ip_address": "192.168.1.1",
"user_agent": "curl/7.68.0",
"created_at": "2025-11-30T12:00:00Z"
}
],
"meta": {
"total": 100,
"limit": 20,
"offset": 0,
"timestamp": "2025-11-30T12:00:00Z"
}
}

Note: Passwords and sensitive fields are automatically sanitized in logs (replaced with `***REDACTED***`).

GET /api/v2/voip/logs/:request_id

Description: Get specific log by request_id

Request:

```
curl -X GET https://edgvoip.it/api/v2/voip/logs/req-123456
```

Response: Single log object.

GET /api/v2/voip/logs/stats

Description: Get API log statistics

Query Parameters:
- `start_date` (optional): ISO 8601 date
- `end_date` (optional): ISO 8601 date

Request:

```
curl -X GET "https://edgvoip.it/api/v2/voip/logs/stats?star
```

Response:
json
{
"success": true,
"data": {
"total_requests": 1000,
"success_count": 950,
"error_count": 40,
"warning_count": 10,
"avg_execution_time_ms": 45,
"success_rate": 95.0
}
}

Webhooks API

Overview

Webhooks allow W3 Suite to receive real-time notifications about events in EDGVoIP (calls, status changes, errors).

Simplified System:
- **Single webhook URL** for all tenants: `https://edgvoip.it/api/webhooks`
- **Webhook secret** configured per tenant in Developer Settings
- **Tenant identification** via `tenant_external_id` in payload
- **HMAC signature authentication** via `X-Webhook-Signature` header (HMAC-SHA256)

Configuration

Webhook Secret Management:
1. Navigate to: `https://edgvoip.it/{tenantSlug}/developer`
2. Go to **Integration** tab
3. Click **Generate** or **Regenerate** webhook secret
4. **Copy the secret immediately** - it's shown only once!
5. Store it securely in your application (W3 Suite)

Webhook Endpoint:
- **URL:** `https://edgvoip.it/api/webhooks` (fixed, same for all tenants)
- Configure this URL in your application (W3 Suite) to receive webhooks
- The `tenant_external_id` in the payload identifies which tenant the event belongs to

Webhook Delivery

HTTP Method: POST
Content-Type: application/json
URL: `https://edgvoip.it/api/webhooks` (fixed)

Headers:
- `X-Webhook-Signature` : HMAC-SHA256 signature of the payload (format: `sha256=`)
- `X-Webhook-Event` : Event type (e.g., `call.ended`)
- `X-Request-ID` : Delivery ID (UUID)
- `X-Tenant-ID` : Tenant UUID (internal)

Authentication:
Your application must verify the HMAC signature using your tenant's webhook secret. The signature is calculated using HMAC-SHA256 of the raw JSON payload.

Important: Since the webhook URL is the same for all tenants, you must:
1. Read `tenant_external_id` from the payload to identify the tenant
2. Retrieve the webhook secret for that tenant from your database
3. Calculate the expected HMAC signature
4. Compare it with the signature in the `X-Webhook-Signature` header

Example Verification (Node.js):
javascript
```
const crypto = require('crypto');

async function verifyWebhookSignature(req, res, next) {
try {
// 1. Get the raw payload (must be string, not parsed JSON)
const rawPayload = JSON.stringify(req.body);

// 2. Extract tenant_external_id from payload to identify tenant
const tenantExternalId = req.body.tenant_external_id;

if (!tenantExternalId) {
return res.status(400).json({ error: 'Missing tenant_external_id' });
}

// 3. Retrieve webhook secret for this tenant from your database
const tenant = await db.query(
'SELECT webhook_secret FROM tenants WHERE tenant_external_id = $1',
[tenantExternalId]
);

if (!tenant || !tenant.webhook_secret) {
return res.status(404).json({ error: 'Tenant not found' });
}

const tenantSecret = tenant.webhook_secret;

// 4. Calculate expected HMAC signature
```

```
const expectedSignature = crypto
.createHmac('sha256', tenantSecret)
.update(rawPayload)
.digest('hex');

// 5. Get received signature from header
const receivedSignature = req.headers['x-webhook-signature']
?.replace('sha256=', '');

if (!receivedSignature) {
return res.status(401).json({ error: 'Missing signature' });
}

// 6. Compare signatures using timing-safe comparison
if (!crypto.timingSafeEqual(
Buffer.from(expectedSignature, 'hex'),
Buffer.from(receivedSignature, 'hex')
)) {
return res.status(401).json({ error: 'Invalid signature' });
}

next(); // Signature verified
} catch (error) {
console.error('Verification error:', error);
res.status(500).json({ error: 'Verification failed' });
}
}

// Apply to webhook endpoint
app.post('/api/webhooks',
express.json(),
verifyWebhookSignature,
(req, res) => {
const { tenant_external_id, type, data } = req.body;
// Process webhook...
res.json({ success: true });
}
);
```

Webhook Events & Payloads

Available Events

Event	Description	Triggered When
`call.start`	Call initiated	CHANNEL_CREATE event in FreeSWITCH
`call.ringing`	Call is ringing	(Future implementation)
`call.answered`	Call answered	CHANNEL_ANSWER event
`call.hold`	Call put on hold	(Future implementation)
`call.unhold`	Call resumed from hold	(Future implementation)
`call.transfer`	Call transferred	(Future implementation)
`call.ended`	Call ended	CHANNEL_HANGUP event
`trunk.status`	Trunk status changed	Health check detects status change
`extension.status`	Extension registration changed	Registration status changes
`api.error`	API error occurred	API request returns status >= 400

Webhook Secret Verification

Simple Header Comparison:

HMAC Signature Verification (Dynamic Multi-Tenant)

How It Works:
Since all tenants use the same webhook URL (`https://edgvoip.it/api/webhooks`), the system uses HMAC-SHA256 signatures to authenticate webhooks. Each tenant has its own webhook secret, and the signature is calculated using that tenant's secret.

Verification Flow:
1. **Receive webhook** at fixed URL: `https://edgvoip.it/api/webhooks`
2. **Extract `tenant_external_id`** from the JSON payload
3. **Lookup tenant** in your database using `tenant_external_id`
4. **Retrieve webhook secret** for that tenant
5. **Calculate expected HMAC** using the tenant's secret and raw payload
6. **Compare** with `X-Webhook-Signature` header using timing-safe comparison

Complete Example (Express.js):

javascript

```
const express = require('express');
const crypto = require('crypto');
const app = express();

// Middleware to verify webhook signature
async function verifyWebhookHMAC(req, res, next) {
try {
// Get raw payload (before JSON parsing)
const rawBody = JSON.stringify(req.body);

// Extract tenant identifier
const tenantExternalId = req.body.tenant_external_id;
if (!tenantExternalId) {
return res.status(400).json({ error: 'Missing tenant_external_id' });
}

// Find tenant and get secret
const tenant = await db.query(
'SELECT webhook_secret FROM tenants WHERE tenant_external_id = $1',
[tenantExternalId]
);

if (!tenant || !tenant.webhook_secret) {
return res.status(404).json({ error: 'Tenant not found' });
}

// Calculate expected signature
const expectedSig = crypto
.createHmac('sha256', tenant.webhook_secret)
.update(rawBody)
.digest('hex');

// Get received signature
const receivedSig = req.headers['x-webhook-signature']?.replace('sha256=',
'');

if (!receivedSig) {
return res.status(401).json({ error: 'Missing signature' });
}
```

```
// Timing-safe comparison
if (!crypto.timingSafeEqual(
Buffer.from(expectedSig, 'hex'),
Buffer.from(receivedSig, 'hex')
)) {
return res.status(401).json({ error: 'Invalid signature' });
}

next(); // Signature verified
} catch (error) {
console.error('Verification error:', error);
res.status(500).json({ error: 'Verification failed' });
}
}

// Webhook endpoint
app.post('/api/webhooks',
express.json(),
verifyWebhookHMAC,
(req, res) => {
const { tenant_external_id, type, data } = req.body;
console.log( Webhook from tenant ${tenant_external_id}:
${type} );
// Process webhook...
res.json({ success: true });
}
);
```

Security Best Practices:
- Always use `crypto.timingSafeEqual()` for signature comparison
- Store webhook secrets securely (encrypted at rest)
- Log failed verification attempts for security monitoring
- Return generic error messages to prevent information leakage

Event Payloads

call.start

Triggered: When a call is initiated

Payload:
json
{
"type": "call.start",
"tenant_id": "550e8400-e29b-41d4-a716-446655440001",
"tenant_external_id": "w3suite-001",
"timestamp": "2025-11-30T12:00:00Z",
"data": {
"call_uuid": "550e8400-e29b-41d4-a716-446655440010",
"caller_id_number": "+39123456789",
"caller_id_name": "John Doe",
"destination_number": "2000",
"call_direction": "inbound",
"context": "demo.edgvoip.it"
}
}

call.answered

Triggered: When a call is answered

Payload:
json
{
"type": "call.answered",
"tenant_id": "550e8400-e29b-41d4-a716-446655440001",
"tenant_external_id": "w3suite-001",
"timestamp": "2025-11-30T12:00:05Z",
"data": {
"call_uuid": "550e8400-e29b-41d4-a716-446655440010",
"caller_id_number": "+39123456789",
"caller_id_name": "John Doe",
"destination_number": "2000",
"call_direction": "inbound"
}
}

call.ended

Triggered: When a call ends

Payload:
json
{
"type": "call.ended",
"tenant_id": "550e8400-e29b-41d4-a716-446655440001",
"tenant_external_id": "w3suite-001",
"timestamp": "2025-11-30T12:05:00Z",
"data": {
"call_uuid": "550e8400-e29b-41d4-a716-446655440010",
"call_direction": "inbound",
"caller_id_number": "+39123456789",
"destination_number": "2000",
"start_time": "2025-11-30T12:00:00Z",
"answer_time": "2025-11-30T12:00:05Z",
"end_time": "2025-11-30T12:05:00Z",
"duration": 300,
"bill_sec": 295,
"hangup_cause": "NORMAL_CLEARING",
"hangup_disposition": "answered"
}
}

trunk.status

Triggered: When trunk registration status changes

Payload:
json
{
"type": "trunk.status",
"tenant_id": "550e8400-e29b-41d4-a716-446655440001",
"tenant_external_id": "w3suite-001",
"timestamp": "2025-11-30T12:00:00Z",
"data": {
"trunk_id": "550e8400-e29b-41d4-a716-446655440000",
"trunk_name": "Warian Trunk",
"previous_status": "unregistered",
"current_status": "registered"

```
}
}
```

extension.status

Triggered: When extension registration status changes

Payload:
```
json
{
"type": "extension.status",
"tenant_id": "550e8400-e29b-41d4-a716-446655440001",
"tenant_external_id": "w3suite-001",
"timestamp": "2025-11-30T12:00:00Z",
"data": {
"extension_id": "550e8400-e29b-41d4-a716-446655440000",
"extension": "2000",
"previous_status": "unregistered",
"current_status": "registered",
"connection_type": "sip",
"registration_ip": "192.168.1.100",
"registration_port": 5060
}
}
```

api.error

Triggered: When an API request fails (status >= 400)

Payload:
```
json
{
"type": "api.error",
"tenant_id": "550e8400-e29b-41d4-a716-446655440001",
"tenant_external_id": "w3suite-001",
"timestamp": "2025-11-30T12:00:00Z",
"data": {
"request_id": "req-123456",
"endpoint": "/api/v2/voip/trunks/invalid-id",
```

"method": "GET",
"status_code": 404,
"error_code": "TRUNK_NOT_FOUND",
"error_message": "Trunk not found",
"api_key_id": "550e8400-e29b-41d4-a716-446655440002"
}
}

Webhook Retry Logic

Retry Strategy: Exponential backoff
- Attempt 1: Immediate
- Attempt 2: 2 seconds delay
- Attempt 3: 4 seconds delay
- Attempt 4: 8 seconds delay

Max Retries: Configurable per webhook (default: 3)

Circuit Breaker: After 10 consecutive failures, webhook is disabled for 5 minutes.

Response Requirements:
- Your webhook endpoint must return HTTP 200-299 for success
- Any other status code triggers retry
- Timeout: 5 seconds (configurable)

Health Check API

Base Endpoint
`/api/v2/voip/health`

GET /api/v2/voip/health

Description: Check API health status

Authentication: Optional (no API key required, but can be used)

Request:

```
curl -X GET https://edgvoip.it/api/v2/voip/health   -H "Con
```

Response:
json
{
"success": true,
"data": {
"status": "healthy",
"timestamp": "2025-11-30T12:00:00Z",
"version": "2.0.0",
"api_version": "v2"
}
}

CDR API

Base Endpoint
`/api/v2/voip/cdr`

GET /api/v2/voip/cdr

Description: List CDR (Call Detail Records) with filtering and pagination

Authentication: `voip:read` scope required

Query Parameters:
- `start_date` (optional): ISO 8601 date
- `end_date` (optional): ISO 8601 date
- `call_direction` (optional): `inbound` | `outbound`
- `limit` (optional, default: 50): Number of results
- `offset` (optional, default: 0): Pagination offset

Request:

```
curl -X GET "https://edgvoip.it/api/v2/voip/cdr?start_date=
```

Response:
json

```
{
"success": true,
"data": [
{
"id": "550e8400-e29b-41d4-a716-446655440000",
"tenant_id": "550e8400-e29b-41d4-a716-446655440001",
"call_uuid": "550e8400-e29b-41d4-a716-446655440010",
"call_direction": "inbound",
"call_type": "sip",
"caller_id_number": "+39123456789",
"caller_id_name": "John Doe",
"destination_number": "2000",
"start_time": "2025-11-30T12:00:00Z",
"answer_time": "2025-11-30T12:00:05Z",
"end_time": "2025-11-30T12:05:00Z",
"duration": 300,
"bill_sec": 295,
"hangup_cause": "NORMAL_CLEARING",
"hangup_disposition": "answered"
}
],
"meta": {
"total": 100,
"limit": 20,
"offset": 0,
"timestamp": "2025-11-30T12:00:00Z"
}
}
```


GET /api/v2/voip/cdr/:id

Description: Get CDR by call UUID

Authentication: `voip:read` scope required

Request:

```
curl -X GET https://edgvoip.it/api/v2/voip/cdr/550e8400-e29
```

Response: Single CDR object.

GET /api/v2/voip/cdr/stats

Description: Get CDR statistics

Authentication: `voip:read` scope required

Query Parameters:
- `start_date` (optional): ISO 8601 date
- `end_date` (optional): ISO 8601 date

Request:

```
curl -X GET "https://edgvoip.it/api/v2/voip/cdr/stats?start
```

Response:
json
{
"success": true,
"data": {
"total_calls": 1000,
"inbound_calls": 600,
"outbound_calls": 400,
"answered_calls": 950,
"missed_calls": 50,
"total_duration": 300000,

"total_bill_sec": 295000,
"avg_duration": 300,
"avg_bill_sec": 295
},
"meta": {
"timestamp": "2025-11-30T12:00:00Z"
}
}

Account Info API

Base Endpoint
`/api/v2/voip/account`

GET /api/v2/voip/account

Description: Get tenant account information and statistics

Authentication: `voip:read` scope required

Request:

```
curl -X GET https://edgvoip.it/api/v2/voip/account   -H "X-
```

Response:
json
{
"success": true,
"data": {
"tenant": {
"id": "550e8400-e29b-41d4-a716-446655440001",
"name": "Demo Tenant",
"slug": "demo",
"tenant_external_id": "w3suite-tenant-001",
"status": "active",

"created_at": "2025-01-01T00:00:00Z",
"updated_at": "2025-11-30T12:00:00Z"
},
"statistics": {
"trunks": 5,
"extensions": 20,
"stores": 3
}
},
"meta": {
"timestamp": "2025-11-30T12:00:00Z"
}
}

DID API

Base Endpoint
`/api/v2/voip/did`

GET /api/v2/voip/did

Description: List all DID (Direct Inward Dialing) numbers for the tenant

Authentication: `voip:read` scope required

Request:

```
curl -X GET https://edgvoip.it/api/v2/voip/did   -H "X-API-
```

Response:
json
{
"success": true,
"data": [
{

"did_number": "+39123456789",
"trunk_id": "550e8400-e29b-41d4-a716-446655440000",
"trunk_external_id": "550e8400-e29b-41d4-a716-446655440000",
"trunk_name": "Warian Trunk",
"created_at": "2025-01-01T00:00:00Z",
"updated_at": "2025-11-30T12:00:00Z"
}
],
"meta": {
"total": 1,
"timestamp": "2025-11-30T12:00:00Z"
}
}

Note: DID numbers are extracted from trunk `did_config` fields. Each trunk can have multiple DID numbers.

Error Handling

Standard Error Response Format

All errors follow this structure:

```
json
{
"success": false,
"error": "Human-readable error message",
"error_code": "ERROR_CODE",
"request_id": "uuid (if available)"
}
```

HTTP Status Codes

Code	Meaning	When Used
200	Success	GET, PUT requests
201	Created	POST requests (resource created)
400	Bad Request	Validation errors, malformed request

401	Unauthorized	Missing or invalid API Key
403	Forbidden	Insufficient permissions (scope)
404	Not Found	Resource not found
409	Conflict	Duplicate resource (e.g., duplicate external_id)
429	Too Many Requests	Rate limit exceeded
500	Internal Server Error	Server error

Common Error Codes

Error Code	Description	Solution
`VALIDATION_ERROR`	Request validation failed	Check required fields and formats
`STORE_NOT_FOUND`	Store ID not found	Verify store_id exists
`TRUNK_NOT_FOUND`	Trunk external_id not found	Verify external_id
`EXTENSION_NOT_FOUND`	Extension external_id not found	Verify external_id
`DUPLICATE_EXTERNAL_ID`	external_id already in use	Use different external_id
`RATE_LIMIT_EXCEEDED`	Too many requests	Wait for rate limit window to reset
`MISSING_API_KEY_CONTEXT`	API Key context missing	Verify X-API-Key header

Error Response Examples

401 Unauthorized:
json
{
"success": false,
"error": "Authentication required: X-API-Key header missing"
}

403 Forbidden:
json
{
"success": false,
"error": "Insufficient permissions for this API Key"
}

404 Not Found:
json
{
"success": false,
"error": "Trunk not found",
"error_code": "TRUNK_NOT_FOUND"
}

429 Rate Limit:
json
{
"success": false,
"error": "Too many requests",
"error_code": "RATE_LIMIT_EXCEEDED",
"retry_after": 1701350400
}

Rate Limiting

Limits

Per API Key: 1000 requests per 15 minutes

Rate Limit Headers

All API responses include rate limit headers:

X-RateLimit-Limit: 1000
X-RateLimit-Remaining: 999
X-RateLimit-Reset: 1701350400

Header Fields:
- `X-RateLimit-Limit` : Maximum requests allowed
- `X-RateLimit-Remaining` : Remaining requests in current window
- `X-RateLimit-Reset` : Unix timestamp when limit resets

Handling Rate Limits

When rate limit is exceeded (429 status):

1. Check `X-RateLimit-Reset` header
2. Wait until reset time
3. Retry request

Example:
javascript
if (response.status === 429) {
const resetTime = parseInt(response.headers.get('X-RateLimit-Reset'));
const waitTime = (resetTime * 1000) - Date.now();
await new Promise(resolve => setTimeout(resolve, waitTime));
// Retry request
}

Database Schema for W3 Suite

Required Fields

To ensure maximum compatibility with EDGVoIP API v2, create these fields in your W3 Suite database:

Tenants Table

sql
ALTER TABLE tenants ADD COLUMN tenant_external_id VARCHAR(255) UNIQUE;
CREATE INDEX idx_tenants_external_id ON tenants(tenant_external_id);

Field Details:
- `tenant_external_id` : Custom alphanumeric identifier (alphanumeric, dots `.` , hyphens `-` , underscores `_` only)
- Must be unique across all tenants
- Used in all API requests to identify tenant

Stores Table

sql
ALTER TABLE stores ADD COLUMN store_id VARCHAR(255);
CREATE INDEX idx_stores_store_id ON stores(store_id);

****Field Details:****
- `store_id` **: Custom identifier for stores (VARCHAR, any format)**
- **Used to link trunks and extensions to stores**

SIP Trunks Table

sql
ALTER TABLE sip_trunks ADD COLUMN external_id UUID;
CREATE UNIQUE INDEX idx_trunks_external_id_tenant ON sip_trunks(tenant_id, external_id);
CREATE INDEX idx_trunks_external_id ON sip_trunks(external_id);

****Field Details:****
- `external_id` **: UUID for API relationships**
- **Must be unique per tenant**
- **Used in API URLs:** `/api/v2/voip/trunks/:external_id`

Extensions Table

sql
ALTER TABLE extensions ADD COLUMN external_id UUID;
CREATE UNIQUE INDEX idx_extensions_external_id_tenant ON extensions(tenant_id, external_id);
CREATE INDEX idx_extensions_external_id ON extensions(external_id);

****Field Details:****
- `external_id` **: UUID for API relationships**
- **Must be unique per tenant**
- **Used in API URLs:** `/api/v2/voip/extensions/:external_id`

Complete Schema Example

sql

```
-- Tenants
CREATE TABLE IF NOT EXISTS tenants (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tenant_external_id VARCHAR(255) UNIQUE,
name VARCHAR(255) NOT NULL,
-- ... other fields
);

-- Stores
CREATE TABLE IF NOT EXISTS stores (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tenant_id UUID NOT NULL REFERENCES tenants(id),
store_id VARCHAR(255),
name VARCHAR(255) NOT NULL,
-- ... other fields
);

-- SIP Trunks
CREATE TABLE IF NOT EXISTS sip_trunks (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tenant_id UUID NOT NULL REFERENCES tenants(id),
store_id UUID REFERENCES stores(id),
external_id UUID,
name VARCHAR(255) NOT NULL,
-- ... other fields
);

CREATE UNIQUE INDEX idx_trunks_external_id_tenant ON
sip_trunks(tenant_id, external_id);

-- Extensions
CREATE TABLE IF NOT EXISTS extensions (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
tenant_id UUID NOT NULL REFERENCES tenants(id),
store_id UUID REFERENCES stores(id),
external_id UUID,
extension VARCHAR(20) NOT NULL,
display_name VARCHAR(255) NOT NULL,
-- ... other fields
);
```

CREATE UNIQUE INDEX idx_extensions_external_id_tenant ON extensions(tenant_id, external_id);

Complete Integration Example (W3 Suite → EDGVoIP)

Step 1: Configure Tenant External ID

1. Login to EDGVoIP: `https://edgvoip.it/{tenantSlug}/developer`
2. Go to **Integration** tab
3. Set `tenant_external_id` (e.g., `w3suite-tenant-001`)
4. Save

Step 2: Create API Key

1. Go to **API Keys** tab
2. Click **Generate New Key**
3. Name: "W3 Suite Integration"
4. Select scopes:
- `voip:read`
- `voip:write`
- `extensions:write`
- `extensions:sync`
- `trunks:sync`
- `stores:read`
- `stores:sync`
5. Copy the key: `sk_live_...`

Step 3: Sync Stores

```
curl -X POST https://edgvoip.it/api/v2/voip/stores/sync   -
    "tenant_external_id": "w3suite-tenant-001",
    "force": false
  }'
```

Step 4: Sync Trunks

```
curl -X POST https://edgvoip.it/api/v2/voip/trunks/sync   -
    "tenant_external_id": "w3suite-tenant-001",
    "force": false
  }'
```

Step 5: Sync Extensions

```
curl -X POST https://edgvoip.it/api/v2/voip/extensions/sync
    "tenant_external_id": "w3suite-tenant-001",
    "force": false
  }'
```

Step 6: Configure Webhooks

```
curl -X POST https://edgvoip.it/api/v2/voip/webhooks   -H "
    "name": "W3 Suite Webhook",
    "url": "https://w3suite.com/webhooks/edgvoip",
    "events": [
      "call.start",
      "call.answered",
      "call.ended",
      "trunk.status",
      "extension.status"
    ],
    "secret": "your-secret-key-here"
  }'
```

Step 7: Verify Webhook Signature

```javascript
const crypto = require('crypto');

app.post('/webhooks/edgvoip', (req, res) => {
const signature = req.headers['x-webhook-signature'];
const secret = 'your-secret-key-here';
const payload = JSON.stringify(req.body);

const expectedSignature = crypto
.createHmac('sha256', secret)
.update(payload)
.digest('hex');

if (signature !== expectedSignature) {
return res.status(401).send('Invalid signature');
}

// Process webhook event
const event = req.body;
console.log('Received event:', event.type, event.data);

res.status(200).send('OK');
});
```

Best Practices

1. API Key Security

- **Never commit API keys to version control**
- **Store keys in environment variables**
- **Rotate keys periodically**
- **Use different keys for different environments** (dev, staging, production)

2. Error Handling

- **Always check `success` field** in responses
- **Handle rate limiting** (429 status)

- **Implement retry logic** with exponential backoff
- **Log all errors** with `request_id` for debugging

3. Webhook Implementation

- **Verify HMAC signature** on every webhook request
- **Return HTTP 200 quickly** (process asynchronously)
- **Implement idempotency** (check `request_id` to prevent duplicate processing)
- **Handle retries gracefully** (webhooks may be delivered multiple times)

4. Sync Strategy

- **Sync on-demand** (manual trigger, not automatic polling)
- **Use `external_id` consistently** across systems
- **Handle conflicts** (EDGVoIP wins, last_updated strategy)
- **Track sync status** in your database

5. Performance

- **Use pagination** for large lists (limit/offset)
- **Cache status data** (trunks/extensions status)
- **Batch operations** when possible
- **Monitor rate limits** (check headers)

Support & Troubleshooting

Common Issues

1. 401 Unauthorized
- Check `X-API-Key` header is present
- Verify API key is valid and not revoked
- Ensure API key hasn't expired

2. 403 Forbidden
- Check API key has required scope
- Verify tenant ownership

3. 404 Not Found
- Verify `external_id` exists

- Check `tenant_external_id` matches API key tenant

4. 429 Rate Limit
- Check `X-RateLimit-Reset` header
- Implement request queuing
- Use multiple API keys if needed (not recommended)

5. Webhook Not Received
- Check webhook is enabled
- Verify URL is accessible (HTTPS required)
- Check circuit breaker status
- Review delivery logs

Debugging

Check API Logs:

```
curl -X GET "https://edgvoip.it/api/v2/voip/logs?status=err
```

Check Webhook Deliveries:

```
curl -X GET "https://edgvoip.it/api/v2/voip/webhooks/{webho
```

Check Status:

```
curl -X GET https://edgvoip.it/api/v2/voip/status/trunks
```

Changelog

Version 2.0 (2025-11-30)
- Initial API v2 release
- API Key authentication
- Complete CRUD operations for trunks/extensions
- Bidirectional sync
- Webhook system
- Real-time status monitoring
- API logging

Contact

For integration support, contact EDGVoIP support team.

Documentation Version: 2.0
Last Updated: 2025-11-30